

**ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING
DEPARTMENT**

CENG 408

Experiments & Results

TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Introduction.....	3
1.1. Purpose of This Document.....	3
2. Disease Detection Module.....	3
2.1. Evaluation Philosophy.....	3
2.2. Experimental Design Principles.....	3
2.2.1. Real-World-Oriented Evaluation Strategy.....	3
2.2.2. Fair Model Comparison Protocol.....	4
2.2.3. Separation of Benchmark vs Deployment Experiments.....	4
2.3. Dataset and Data Pipeline.....	4
2.3.1. Dataset Sources.....	4
2.3.2. Dataset Construction Strategy.....	4
2.3.3. Data Cleaning and Validation.....	5
2.3.4. Data Preprocessing Pipeline.....	5
2.3.5. Data Augmentation Strategy.....	5
2.3.6. Dataset Splitting Strategy.....	5
2.4. Model Architectures.....	6
2.4.1. CNN-Based Models.....	6
2.4.2. Transformer-Based Models.....	6
2.4.3. Architectural Differences.....	6
2.5. Training Configuration.....	6
2.5.1. Optimization Strategy.....	6
2.5.2. Sampling Strategy.....	6
2.5.3. Hyperparameters.....	7
2.6. Evaluation Metrics.....	7
2.6.1. Classification Metrics.....	7
2.6.2. Metric Justification.....	7
2.6.3. Validation Selection Metric.....	7
2.7. Experimental Results.....	7
2.7.1. Single-Backbone Benchmark.....	7
2.7.2. Deployed Models.....	8
2.7.3. Per-Class Performance Analysis.....	9
2.7.4. Confusion Patterns.....	9
2.8. Generalization and Robustness Analysis.....	10
2.8.1. Lab-to-Field Gap.....	10
2.8.2. Class Stability.....	10
2.8.3. Failure Case Analysis.....	10
3. LLM Module Evaluation.....	11
3.1. Evaluation Methodology.....	11
3.2. Scenario-Based Testing.....	11
3.3. Tool Usage Analysis.....	12

3.4. Latency Analysis.....	13
3.5. Reliability and Grounding.....	13
3.6. Failure Cases.....	14

1. Introduction

1.1. Purpose of This Document

This document presents the experimental evaluation of the machine learning components in the TARAS platform. It reports the methodology, evidence, and observed performance of the models under both controlled and deployment-oriented conditions.

The evaluation covers two subsystems: the vision-based disease detection module and the LLM-based advisory module. Since these address different tasks, they are evaluated differently: the disease module with quantitative classification metrics, and the advisory module with qualitative, scenario-based testing.

2. Disease Detection Module

2.1. Evaluation Philosophy

The disease classifier is judged on real-world (field) performance, not laboratory accuracy. Models trained on controlled datasets like PlantVillage reach 99%+ on similar lab images, but that does not transfer to field photos with uneven lighting, cluttered backgrounds, and partial occlusion.

The test set is therefore sliced by source. The field slice (PlantDoc photographs, 332 images) is the primary deployment benchmark. The lab slice (PlantVillage, 1,594 images) is a controlled reference that is already near-saturated (>0.99 macro F1), so it is not a meaningful discriminator between models. All model-selection decisions are driven by the field slice.

2.2. Experimental Design Principles

2.2.1. Real-World-Oriented Evaluation Strategy

Field images are the hard, deployment-relevant distribution. The field slice (PlantDoc, 332 images) is the primary benchmark; the lab slice (PlantVillage, 1,594 images) is a secondary, near-saturated reference. Reporting both makes the lab-to-field gap explicit instead of hiding real-world difficulty behind a high lab score.

2.2.2. Fair Model Comparison Protocol

Every candidate backbone uses the same split, preprocessing, augmentation, optimizer, and metrics. Performance differences

therefore reflect architecture and input resolution, not training conditions.

2.2.3. Separation of Benchmark vs Deployment Experiments

The project ships two models from the same pipeline:

- A cloud (server) model optimized for maximum accuracy, used for server-side inference.
- A mobile model optimized for size and latency, run on-device.

Single backbones are benchmarked first. The strongest are combined into an ensemble for the server, and a compact student is distilled from a strong teacher for mobile.

2.3. Dataset and Data Pipeline

2.3.1. Dataset Sources

The dataset spans 14 disease and healthy classes across 7 crops (tomato, potato, corn, pepper, cherry, peach, squash), drawn from three sources:

- **PlantVillage (lab):** 33-35k controlled-lighting, single-leaf images. Easy distribution.
- **PlantDoc (field):** 1.5k uncontrolled field photos with complex backgrounds. The production-relevant, hard distribution.
- **Real-world blend:** 40-74 extra field images, mostly covering spider_mites, which PlantDoc does not include.

Combining lab and field sources is what lets the model learn clear symptoms from clean images while staying robust on real photos. Domain shift between these sources is the central challenge of the task.

2.3.2. Dataset Construction Strategy

The 28,600 training images are heavily lab-dominated: 27,879 PlantVillage (97.5%), 681 PlantDoc (2.4%), and 40 real-world (0.1%). This field scarcity is the key constraint on real-world performance. It is countered with loss-side source weighting (Section 6.2.5.2) rather than by collecting more field data, which was not available.

2.3.3. Data Cleaning and Validation

The combined sources are deduplicated and leakage-audited before splitting. Exact duplicates (pHash and dHash) and near-duplicates (perceptual-hash Hamming distance ≤ 4 within a source) are clustered with a union-find pass, and whole clusters are kept on one side of the split. This guarantees no near-duplicate leaks from training into tests, which would otherwise inflate scores. A label-noise audit also removes images with uncertain or inconsistent labels.

2.3.4. Data Preprocessing Pipeline

All images are resized so the shortest side is 256, then center-cropped to 224x224, converted to tensors, and normalized with ImageNet mean and standard deviation in RGB order. The same preprocessing is used at training and inference so input statistics match.

2.3.5. Data Augmentation Strategy

Augmentation is applied to the training set only. It includes random-resized cropping (scale 0.7-1.0), horizontal flips, color jitter, motion and Gaussian blur, Gaussian noise, JPEG compression, and random shadows. This "field domain randomization" makes the model less dependent on clean lab conditions. Validation and test sets are left unchanged for comparable evaluation.

2.3.6. Dataset Splitting Strategy

The 35,924 images are split 28,600 train / 5,383 validation / 1,941 tests, stratified per class and per source, with the test set balanced to 30-150 images per class. Each test image is tagged by source: 332 fields (PlantDoc), 1,594 lab (PlantVillage), and 15 real-world.

Three classes (mosaic_virus, spider_mites, target_spot) have zero field-test images, because PlantDoc does not cover them, so field macro F1 is computed over the 11 classes that have field coverage. Best-checkpoint selection during training maximizes a composite score.

$$0.5 \times \text{validation macro F1} + 0.5 \times \text{field macro F1}$$

This balances clean-data stability against real-world performance and keeps selection aligned with deployment.

2.4. Model Architectures

2.4.1. CNN-Based Models

ConvNeXt is the convolutional backbone family evaluated in two variants: ConvNeXt-V1-Base and ConvNeXt-V2-Base. CNNs capture local features such as texture, edges, and lesion patterns, which are important for fine-grained disease symptoms. EfficientNet-B0 is also used, but only as the compact student for the mobile model (Section 6.2.7.2), not as a server backbone.

2.4.2. Transformer-Based Models

The Swin Transformer family is evaluated at several capacities and input sizes: Swin-Tiny and Swin-Small at 224, and Swin-Small and Swin-Base at 384. Swin uses shifted-window self-attention to combine local detail with broader leaf-level context. The 384 input size was explored specifically for robustness when the diseased leaf is small in the frame, a scenario flagged for real use.

2.4.3. Architectural Differences

CNNs build spatial hierarchies through stacked convolutions and excel at local texture. Swin models share information across regions through windowed attention, which helps when symptoms are subtle or spread out. Larger input (384) and higher capacity (Base) generally improve the field slice at the cost of size and latency. Comparing both families under identical conditions determines which is most suitable per deployment target.

2.5. Training Configuration

2.5.1. Optimization Strategy

Training is two-stage. Stage 1 is a short head-only warmup with the backbone frozen, letting the new classifier converge. Stage 2 fine-tunes the full network with AdamW (lower learning rate on the body than the head) for up to 50 epochs, with early stopping on the composite score. The loss is cross-entropy with a 0.5 weight on lab samples, so field samples receive proportionally more gradient. An exponential moving average (EMA) of the weights is tracked, and the EMA checkpoint is often selected.

2.5.2. Sampling Strategy

Because field data is only 2.4% of training, balancing is done on the loss side rather than by resampling. Real-world samples are upweighted ($\times 5$), and one experiment upweighted PlantDoc ($\times 8$) to raise its effective share. Explicit per-class weighting was tested

but consistently hurt results, so it was not used in the final models.

2.5.3. Hyperparameters

All backbones share one configuration so differences reflect architecture, not tuning. Input resolution is 224 or 384 depending on the variant, batch size is set by hardware limits, and Stage 2 runs to convergence with patience-based early stopping. Horizontal-flip test-time augmentation is applied at evaluation.

2.6. Evaluation Metrics

2.6.1. Classification Metrics

The module is evaluated with accuracy, macro F1, and weighted F1, reported on the full test set and on the field and lab slices separately. Macro F1 is the primary metric because it weights every class equally and exposes weak minority classes that accuracy would hide. Accuracy and weighted F1 are reported as secondary contexts.

2.6.2. Metric Justification

Scores are always separated into the field slice (PlantDoc, the 11 field-present classes) and the lab slice (PlantVillage). The field slice is the deployment benchmark; the lab slice is a saturated control. Keeping them separate prevents the large, easy lab set from masking real-world performance in a single blended number.

2.6.3. Validation Selection Metric

Checkpoints and ensembles are selected on validation, not test, to keep reported numbers honest. Selection maximizes $0.5 \times \text{validation macro F1} + 0.5 \times \text{field macro F1}$. Tuning on the test set was measured to inflate the field score by roughly 2 points, so it is avoided for all reported figures.

2.7. Experimental Results

2.7.1. Single-Backbone Benchmark

All backbones were trained under the identical recipe and compared on the test set ($n = 1,941$). Lab F1 is near-saturated for every model (~ 0.99); the field slice is the real discriminator. Higher input resolution (384) and field oversampling give the best single-model field F1.

Backbone	Input	Macro F1	Field F1	Lab F1	Params
Swin-Base (field-oversampled)	224	0.9678	0.8393	0.9935	88M
Swin-Small	384	0.9678	0.8348	0.9945	49M
Swin-Base	384	0.9696	0.8332	0.9968	87M
ConvNeXt-V1-Base	224	0.9678	0.8290	0.9964	88M
Swin-Small	224	0.9649	0.8224	0.9943	49M
Swin-Base	224	0.9665	0.8204	0.9963	88M
ConvNeXt-V2-Base	224	0.9594	0.8042	0.9900	88M

2.7.2. Deployed Models

- Server ensemble:** The deployed cloud model is a three-way, equal-weight ensemble of ConvNeXt-V2-Base, Swin-Small (384), and the field-oversampled Swin-Base (224), chosen by a brute-force subset search on validation. It reaches field F1 0.8488, macro F1 0.9697, lab F1 0.9944, and a worst-class F1 of 0.6486, with temperature calibration applied at inference (expected calibration error reduced from roughly 8-10% to under 2%). Notably, the ensemble outperforms every single backbone on the field slice and on worst-class F1, and the subset search beats a larger 5-way ensemble while being smaller and faster, because the third member (ConvNeXt-V2-Base) adds diversity rather than raw single-model strength.

Metric	Value
Accuracy	0.9701
Macro F1	0.9697
Field F1	0.8488
Lab F1	0.9944
Worst-class F1	0.6486

The mobile model trails the server ensemble on the field slice (0.79 vs 0.85), which is the expected cost of compressing a large teacher into a 7.7 MB student.

2.7.3. Per-Class Performance Analysis

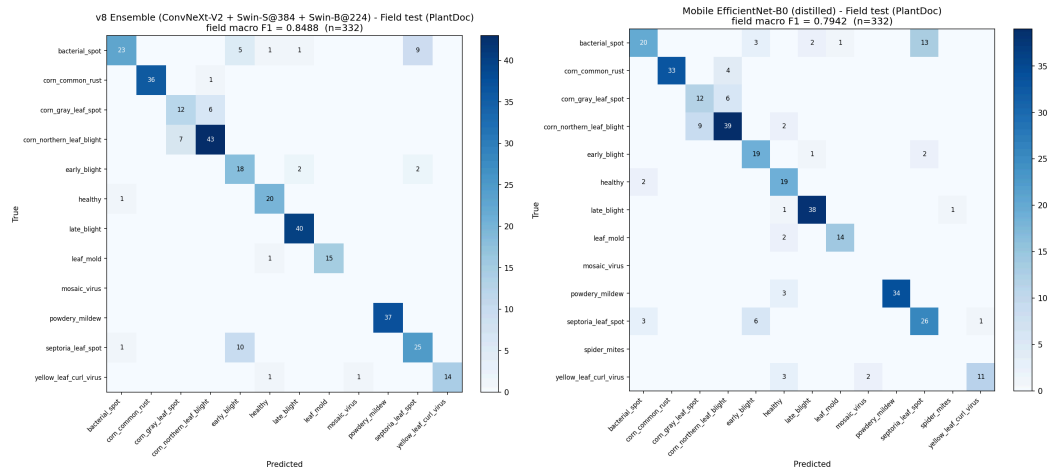
Per-class field F1 for the server ensemble is shown below, over the 11 classes present in the field slice. The three classes without field-test images (mosaic_virus, spider_mites, target_spot) are evaluated only on the lab slice and are excluded here.

Class	Field images	Field F1
powdery_mildew	37	1.000
corn_common_rust	37	0.986
leaf_mold	16	0.968
late_blight	40	0.964
yellow_leaf_curl_virus	16	0.933
healthy	21	0.909
corn_northern_leaf_blight	50	0.860
bacterial_spot	39	0.719
septoria_leaf_spot	36	0.694
early_blight	22	0.655
corn_gray_leaf_spot	18	0.649

Classes with distinctive symptoms (powdery_mildew, rust, leaf_mold) are essentially solved on field images. The weak classes are the visually overlapping leaf-spot and blight groups: bacterial_spot, septoria_leaf_spot, early_blight, and corn_gray_leaf_spot, the last of which also has the fewest field-test images (18).

2.7.4. Confusion Patterns

Errors concentrate on visually similar classes rather than spreading randomly. The dominant confusion is bacterial_spot with septoria_leaf_spot, two tomato leaf-spot diseases with near-identical lesions. Early_blight and late_blight are confused with each other, as are the two corn diseases, corn_northern_leaf_blight and corn_gray_leaf_spot. These pairings explain the low field F1 of those classes and match the per-class results above.



2.8. Generalization and Robustness Analysis

2.8.1. Lab-to-Field Gap

The clearest robustness signal is the gap between the lab and field slices. Both deployed models score about 0.99 macro F1 on lab images but drop to 0.85 (server ensemble) and 0.79 (mobile) on field images. The gap is driven by the training distribution, where only 2.4% of images are field photos, and by the genuine difficulty of field conditions. The field score is therefore the honest estimate of deployment performance, and the lab score should not be read as real-world accuracy.

2.8.2. Class Stability

Stability across slices is uneven. Distinctive-symptom classes (powdery_mildew, rust, leaf_mold, late_blight) hold high F1 on both lab and field. The fragile classes are the leaf-spot and blight groups (bacterial_spot, septoria_leaf_spot, early_blight, corn_gray_leaf_spot), which are strong in the lab but weak in the field. Mobile-model checkpoint selection deliberately favored the worst-class floor over average F1 to keep these fragile classes from collapsing.

2.8.3. Failure Case Analysis

Most failures come from the visual-similarity confusions described in Section 6.2.7.4. Two structural limitations underlie them. First, field data is scarce (2.4% of training) and absent entirely for three classes, so the model has little real-world signal for the hardest cases. Second, several disease pairs are genuinely ambiguous at the leaf level, especially in early stages or poor lighting. The dataset is at its practical ceiling; further gains would require more field photographs, additional label cleaning, or hard-negative

mining focused on the bacterial_spot and septoria_leaf_spot pair.

3. LLM Module Evaluation

3.1. Evaluation Methodology

The advisory module is evaluated qualitatively, not with the classification metrics used for the disease module. An advisory reply is open-ended text with no single correct answer, so accuracy and F1 do not apply.

Each response is judged on five points: factual correctness against live sensor data, correct tool selection, brevity, language fidelity (Turkish or English in Latin script), and no fabrication.

The module is agentic. For each question the model receives a cached system prompt, the farm and field inventory, and the last ten messages of the session, then runs a tool-use loop (reason, call a tool, read the result, reason again) for up to eight iterations within a 30-second budget. Evaluation exercises this full production loop, not raw model output.

Because the loop is model-portable, several candidate models were tested under identical conditions before one was chosen. Three procedures were used

- Functional validation of the tool loop against representative scenarios.
- A 25-run stability test per model, using a set of test prompts we wrote.
- A 7-prompt multi-turn quality test across a realistic conversation.

3.2. Scenario-Based Testing

Testing covered the four question types the assistant must handle in the field:

- **Live-data queries ("how is my field?", "should I irrigate Zone 2?"):** call a data tool, then answer in text leading with the number.
- **Agronomic-theory questions (crop coefficients, soil types, irrigation efficiency):** answered from the built-in reference, with a knowledge-base search only when the topic is not covered.
- **Navigation requests ("show me the moisture chart"):** a single navigation call followed by a one-sentence prompt on the target screen.
- **Out-of-scope or data-absent questions:** decline in one sentence without inventing a value.

Several language models were evaluated against these scenarios under

the identical production loop before one was chosen. Claude Haiku 4.5 (Anthropic) was selected as the production model. As a frontier model it gave the most reliable agentic behavior: native structured tool calls, consistent adherence to the brevity and language rules, and none of the failure modes that affected the open candidates.

In parallel, a set of open models hosted on Groq was evaluated as a zero-cost alternative. Each candidate was run 25 times against a set of test prompts we wrote to check stability and quality, and scored on the 7-prompt multi-turn test. gpt-oss-120b was clearly the strongest: 93.5% multi-turn quality, the best open-model reasoning (MMLU-Pro 90), and zero language drift, channel leakage, or empty responses across the 25 runs. It is kept as a configurable alternative for cost-sensitive deployment.

The other open candidates were rejected for concrete, reproducible reasons:

- qwen3-32b: leaked non-Latin script, mixing Turkish, Chinese, and Japanese in three of five refusals.
- llama-4-scout: ignored the built-in agronomy reference and over-used the knowledge search.

3.3. Tool Usage Analysis

The assistant exposes eleven tools in three groups:

- **Navigation (1):** opens a specific section of a screen.
- **Data (9):** live PostgreSQL reads for field overview; zone latest readings, history, and configuration; irrigation history; sensor diagnostics; carbon summary; disease history; and active alerts.
- **Knowledge (1):** a full-text search over the agricultural knowledge base.

Tool selection follows an explicit decision tree in the system prompt with a text-first policy. The assistant answers with numbers from a data tool whenever possible, and uses navigation only when seeing a chart, the 3D field, or a camera workflow beats a text reply.

Three behaviors were checked:

- **Correct selection:** A data tool for status questions, not a navigation "for completeness".
- **Parameter discipline:** the exact UUIDs from the per-request context are reused, since a truncated or invented ID breaks the query.
- **Loop efficiency:** the loop is capped at eight iterations, with one tool call per turn unless several zones or ranges are genuinely needed, and never a repeated identical call.

Tool handling was a decisive factor in model selection. Several open candidates failed here specifically, which ruled them out regardless of language

quality.

3.4. Latency Analysis

Latency is driven by the agentic loop, not a single call. Each turn can issue up to eight sequential model calls within a 30-second timeout, though the decision tree keeps most queries to one or two iterations.

Two mechanisms reduce perceived latency:

- The system prompt is served through Anthropic's ephemeral prompt cache, so its large token block is not reprocessed or re-billed every turn.
- The final answer is streamed to the client over Server-Sent Events, and each tool phase emits an "analyzing" status so the interface can show progress.

Request latency and iteration counts are logged on the server. Typical end-to-end response time is three to six seconds. A lighter Groq model (gpt-oss-20b) was observed around 4 seconds end-to-end and can be swapped in when latency matters more than reasoning depth.

3.5. Reliability and Grounding

Grounding is enforced at three levels:

- **Live data:** all readings come from the nine data tools, which query PostgreSQL directly. Stale results are not reused; the assistant re-fetches when asked again.
- **Built-in reference:** agronomic theory is answered from a condensed reference (crop coefficients by stage, soil field-capacity and wilting-point values, irrigation efficiencies, ET formulas) before any external lookup.
- **No fabrication:** when data is missing or a tool fails, the assistant says so in one sentence instead of guessing.

Operational reliability comes from several safeguards. Timeouts and the maximum-iteration limit return a graceful message rather than a partial answer. Advisory requests are rate-limited to three per minute per user for cost control. Model output is sanitized to strip reasoning tags, legacy tool-call text, and reasoning-channel tokens before display.

The tool loop is model-portable. It runs on Claude Haiku 4.5 in production but can switch to a Groq-hosted open model without code changes, so the service is not tied to one provider.

Language safety was a specific concern, since the chat is rendered in Latin script for Turkish users. Across the 25-run test the open alternative (gpt-oss-120b) produced no language drift and no non-Latin output, unlike qwen3-32b, which mixed three scripts in a single refusal. The production model showed no such issues.

3.6. Failure Cases

Evaluation surfaced several failure modes that shaped model selection and design.

The most serious was script mixing. One candidate produced blended Turkish, Chinese, and Japanese text in about three of five refusals, which is unacceptable for a Turkish-facing product.

Tool-protocol failures were also disqualifying: one model bypassed the built-in reference and over-searched, another emitted a deprecated text-based tool-call format, and the compound models did not accept custom tools.

A lower-severity issue affected every open model: a markdown bias, with about 60% of replies opening in a list despite the brevity rules. This is handled by post-processing rather than trusting the model.

Two failure modes remain by design. A query that exceeds eight iterations or the 30-second budget returns a graceful fallback message instead of an incomplete answer. Claude Haiku 4.5, the production model, showed none of the script or tool-format failures seen in the open candidates.